

ExtraCredit2

December 12, 2025

1 DSCC 201/401 Final Project - Question1

L. EXTRA CREDIT 2 (10 points): Use the PyTorch library as provided by the Python 3 (Python 3 (PyTorch 1.9.1)) kernel to build a simple artificial neural network for the prediction of the solar radio flux based on the sunspot number and the individual number of X-ray and optical solar flares. Begin with importing your cleaned data file (solar_data_clean.csv) into a Panadas data frame. Partition the data set appropriately and train your neural network. What is the predicted radio flux for a day when 73 sunspots are observed on the sun and 2 solar flares with C-class energy levels and 1 solar flare with an M-class energy level occur? How confident are you in the answer? Explain your reasoning. Please provide a PDF of your Jupyter notebook showing ALL input and output and upload BOTH the PDF and the Jupyter notebook file (ExtraCredit2.ipynb and ExtraCredit2.pdf) showing all input and output.

```
In [57]: import numpy as np
import pandas as pd

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

import torch
from torch import nn
from torch.utils.data import TensorDataset, DataLoader

# Load the cleaned solar data file
df = pd.read_csv("solar_data_clean.csv")
df.head()
```

```
Out [57]:
```

	Date	Radio_Flux_10.7_cm	SESC_Sunspot_Number	Sunspot_Area	\
0	1997-01-01	72	0	0	
1	1997-01-02	72	0	0	
2	1997-01-03	73	0	0	
3	1997-01-04	74	13	10	
4	1997-01-05	74	15	20	

	New_Regions	GOES15_XRay_Bkgd_Flux	XRay_C	XRay_M	XRay_X	S	Optical_1	\
0	0	A0.5	0	0	0	1	0	
1	0	A0.5	0	0	0	1	0	
2	0	A0.5	0	0	0	1	0	

3	1	A0.6	0	0	0	1	0
4	0	A0.7	0	0	0	1	0

	Optical_2	Optical_3
0	0	0
1	0	0
2	0	0
3	0	0
4	0	0

```
In [58]: # Select feature and target columns
feature_cols = [
    "SESC_Sunspot_Number",
    "XRay_C",
    "XRay_M",
    "XRay_X",
    "S",
    "Optical_1",
    "Optical_2",
    "Optical_3"
]

target_col = "Radio_Flux_10.7_cm"

X = df[feature_cols].to_numpy()
y = df[target_col].to_numpy()

# Partition the data set into training and test sets
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Standardize the features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Convert NumPy arrays to PyTorch tensors
X_train_t = torch.from_numpy(X_train_scaled).float()
y_train_t = torch.from_numpy(y_train).float().view(-1, 1)

X_test_t = torch.from_numpy(X_test_scaled).float()
y_test_t = torch.from_numpy(y_test).float().view(-1, 1)

print("Train tensors:", X_train_t.shape, y_train_t.shape)
print("Test tensors :", X_test_t.shape, y_test_t.shape)
```

```

# Create TensorDataset and DataLoader
train_set = TensorDataset(X_train_t, y_train_t)
test_set = TensorDataset(X_test_t, y_test_t)

train_loader = DataLoader(train_set, batch_size=32, shuffle=True)
test_loader = DataLoader(test_set, batch_size=32, shuffle=False)

Train tensors: torch.Size([8433, 8]) torch.Size([8433, 1])
Test tensors : torch.Size([2109, 8]) torch.Size([2109, 1])

```

```

In [59]: # Build a neural network model
n_features = X_train_t.shape[1]

class SolarNet(nn.Module):
    def __init__(self, n_features):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(n_features, 32),
            nn.ReLU(),
            nn.Linear(32, 16),
            nn.ReLU(),
            nn.Linear(16, 1)
        )

    def forward(self, x):
        return self.net(x)

model = SolarNet(n_features)
print(model)

# Define loss function and optimizer
criterion = nn.MSELoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)

```

```

SolarNet(
  (net): Sequential(
    (0): Linear(in_features=8, out_features=32, bias=True)
    (1): ReLU()
    (2): Linear(in_features=32, out_features=16, bias=True)
    (3): ReLU()
    (4): Linear(in_features=16, out_features=1, bias=True)
  )
)

```

```

In [60]: # Train the neural network model
epochs = 100

```

```

for epoch in range(epochs):

    # Training phase
    model.train()
    train_loss = 0

    for X_batch, y_batch in train_loader:
        optimizer.zero_grad()

        y_pred = model(X_batch)
        loss = criterion(y_pred, y_batch)

        loss.backward()
        optimizer.step()

        train_loss = train_loss + loss.item() * X_batch.size(0)

    train_loss = train_loss / len(train_loader.dataset)

    # Validate phase
    model.eval()
    val_loss = 0
    val_mae = 0

    with torch.no_grad():
        for X_batch, y_batch in test_loader:
            y_pred = model(X_batch)

            mse_loss = criterion(y_pred, y_batch)

            abs_error = torch.abs(y_pred - y_batch)
            mae_loss = torch.mean(abs_error)

            val_loss = val_loss + mse_loss.item() * X_batch.size(0)
            val_mae = val_mae + mae_loss.item() * X_batch.size(0)

    val_loss = val_loss / len(test_loader.dataset)
    val_mae = val_mae / len(test_loader.dataset)
    print("Epoch", epoch + 1, "/", epochs,
          f"Train MSE: {train_loss:.4f} "
          f"Val MSE: {val_loss:.4f} "
          f"Val MAE: {val_mae:.4f}")

```

```

Epoch 1 / 100 Train MSE: 12185.0119 Val MSE: 5401.6982 Val MAE: 66.8959
Epoch 2 / 100 Train MSE: 2008.1520 Val MSE: 870.1762 Val MAE: 20.6937
Epoch 3 / 100 Train MSE: 692.6682 Val MSE: 588.7899 Val MAE: 16.4865
Epoch 4 / 100 Train MSE: 520.9130 Val MSE: 462.7639 Val MAE: 14.4552

```

Epoch 5 / 100	Train MSE: 431.1289	Val MSE: 386.3536	Val MAE: 13.1436
Epoch 6 / 100	Train MSE: 377.4093	Val MSE: 341.7519	Val MAE: 12.3849
Epoch 7 / 100	Train MSE: 344.2419	Val MSE: 313.3672	Val MAE: 11.8798
Epoch 8 / 100	Train MSE: 322.6823	Val MSE: 293.7842	Val MAE: 11.3969
Epoch 9 / 100	Train MSE: 309.4405	Val MSE: 279.9771	Val MAE: 11.1616
Epoch 10 / 100	Train MSE: 298.0881	Val MSE: 270.8099	Val MAE: 10.8698
Epoch 11 / 100	Train MSE: 291.1228	Val MSE: 264.3270	Val MAE: 10.7208
Epoch 12 / 100	Train MSE: 285.7601	Val MSE: 263.7703	Val MAE: 10.7937
Epoch 13 / 100	Train MSE: 282.1133	Val MSE: 256.2530	Val MAE: 10.7123
Epoch 14 / 100	Train MSE: 279.6626	Val MSE: 253.1686	Val MAE: 10.6568
Epoch 15 / 100	Train MSE: 276.3942	Val MSE: 251.5075	Val MAE: 10.5956
Epoch 16 / 100	Train MSE: 274.1610	Val MSE: 246.7913	Val MAE: 10.5253
Epoch 17 / 100	Train MSE: 273.2405	Val MSE: 245.8037	Val MAE: 10.5406
Epoch 18 / 100	Train MSE: 270.5662	Val MSE: 244.3434	Val MAE: 10.5011
Epoch 19 / 100	Train MSE: 269.3798	Val MSE: 243.1903	Val MAE: 10.5679
Epoch 20 / 100	Train MSE: 267.9063	Val MSE: 241.5004	Val MAE: 10.5107
Epoch 21 / 100	Train MSE: 266.0608	Val MSE: 239.8492	Val MAE: 10.4183
Epoch 22 / 100	Train MSE: 264.0870	Val MSE: 241.2311	Val MAE: 10.4912
Epoch 23 / 100	Train MSE: 262.9567	Val MSE: 237.5423	Val MAE: 10.4917
Epoch 24 / 100	Train MSE: 261.0780	Val MSE: 237.7506	Val MAE: 10.3328
Epoch 25 / 100	Train MSE: 259.9238	Val MSE: 235.1808	Val MAE: 10.2697
Epoch 26 / 100	Train MSE: 258.3785	Val MSE: 233.4069	Val MAE: 10.4446
Epoch 27 / 100	Train MSE: 258.0613	Val MSE: 233.0210	Val MAE: 10.1868
Epoch 28 / 100	Train MSE: 255.4583	Val MSE: 229.5177	Val MAE: 10.1826
Epoch 29 / 100	Train MSE: 254.1461	Val MSE: 230.7094	Val MAE: 10.2134
Epoch 30 / 100	Train MSE: 254.4622	Val MSE: 227.6011	Val MAE: 10.2711
Epoch 31 / 100	Train MSE: 251.7141	Val MSE: 226.7092	Val MAE: 10.1676
Epoch 32 / 100	Train MSE: 250.6639	Val MSE: 225.7658	Val MAE: 10.2342
Epoch 33 / 100	Train MSE: 249.6795	Val MSE: 222.7289	Val MAE: 10.1153
Epoch 34 / 100	Train MSE: 248.4909	Val MSE: 222.8928	Val MAE: 10.1874
Epoch 35 / 100	Train MSE: 247.8288	Val MSE: 233.1805	Val MAE: 10.1965
Epoch 36 / 100	Train MSE: 248.1407	Val MSE: 222.7664	Val MAE: 10.3869
Epoch 37 / 100	Train MSE: 246.6501	Val MSE: 221.8652	Val MAE: 10.1569
Epoch 38 / 100	Train MSE: 246.2578	Val MSE: 222.7426	Val MAE: 10.2789
Epoch 39 / 100	Train MSE: 246.1543	Val MSE: 220.3724	Val MAE: 10.0659
Epoch 40 / 100	Train MSE: 246.1181	Val MSE: 220.1692	Val MAE: 10.0986
Epoch 41 / 100	Train MSE: 246.4084	Val MSE: 223.5065	Val MAE: 10.0719
Epoch 42 / 100	Train MSE: 245.0958	Val MSE: 221.4107	Val MAE: 10.1373
Epoch 43 / 100	Train MSE: 243.5192	Val MSE: 223.2182	Val MAE: 10.1694
Epoch 44 / 100	Train MSE: 245.3243	Val MSE: 221.9481	Val MAE: 10.2937
Epoch 45 / 100	Train MSE: 244.1907	Val MSE: 221.0814	Val MAE: 10.2645
Epoch 46 / 100	Train MSE: 244.0301	Val MSE: 220.4837	Val MAE: 10.0692
Epoch 47 / 100	Train MSE: 243.9646	Val MSE: 218.5468	Val MAE: 10.0759
Epoch 48 / 100	Train MSE: 243.1200	Val MSE: 220.2555	Val MAE: 10.1877
Epoch 49 / 100	Train MSE: 243.3577	Val MSE: 220.0011	Val MAE: 10.2044
Epoch 50 / 100	Train MSE: 242.6544	Val MSE: 220.2574	Val MAE: 10.0220
Epoch 51 / 100	Train MSE: 243.2717	Val MSE: 219.9352	Val MAE: 10.1199
Epoch 52 / 100	Train MSE: 242.1496	Val MSE: 225.2659	Val MAE: 10.2602

Epoch 53 / 100	Train MSE: 243.1465	Val MSE: 218.8368	Val MAE: 10.1238
Epoch 54 / 100	Train MSE: 242.3440	Val MSE: 223.7784	Val MAE: 10.2385
Epoch 55 / 100	Train MSE: 242.0923	Val MSE: 219.2396	Val MAE: 10.1399
Epoch 56 / 100	Train MSE: 242.0634	Val MSE: 219.4617	Val MAE: 10.0707
Epoch 57 / 100	Train MSE: 242.3973	Val MSE: 218.1128	Val MAE: 10.1127
Epoch 58 / 100	Train MSE: 241.4750	Val MSE: 226.3682	Val MAE: 10.4192
Epoch 59 / 100	Train MSE: 241.8444	Val MSE: 219.5279	Val MAE: 10.0429
Epoch 60 / 100	Train MSE: 241.3135	Val MSE: 218.6743	Val MAE: 10.0825
Epoch 61 / 100	Train MSE: 240.3461	Val MSE: 221.7535	Val MAE: 10.1798
Epoch 62 / 100	Train MSE: 240.9318	Val MSE: 218.2817	Val MAE: 10.1495
Epoch 63 / 100	Train MSE: 241.2713	Val MSE: 218.7667	Val MAE: 10.0167
Epoch 64 / 100	Train MSE: 241.2945	Val MSE: 226.0114	Val MAE: 10.3931
Epoch 65 / 100	Train MSE: 241.0654	Val MSE: 216.9650	Val MAE: 10.0587
Epoch 66 / 100	Train MSE: 241.2613	Val MSE: 217.0408	Val MAE: 10.0667
Epoch 67 / 100	Train MSE: 242.0279	Val MSE: 224.7860	Val MAE: 10.4801
Epoch 68 / 100	Train MSE: 240.3285	Val MSE: 218.8191	Val MAE: 10.2685
Epoch 69 / 100	Train MSE: 240.6791	Val MSE: 221.6659	Val MAE: 10.2116
Epoch 70 / 100	Train MSE: 240.3145	Val MSE: 220.3427	Val MAE: 10.2588
Epoch 71 / 100	Train MSE: 240.4349	Val MSE: 216.6736	Val MAE: 10.0886
Epoch 72 / 100	Train MSE: 239.5166	Val MSE: 218.4111	Val MAE: 10.0401
Epoch 73 / 100	Train MSE: 239.2177	Val MSE: 217.8989	Val MAE: 10.1431
Epoch 74 / 100	Train MSE: 239.9495	Val MSE: 217.8954	Val MAE: 10.0750
Epoch 75 / 100	Train MSE: 240.0777	Val MSE: 216.9833	Val MAE: 10.0545
Epoch 76 / 100	Train MSE: 238.7862	Val MSE: 216.9785	Val MAE: 10.1029
Epoch 77 / 100	Train MSE: 239.1136	Val MSE: 220.3233	Val MAE: 10.2809
Epoch 78 / 100	Train MSE: 239.5935	Val MSE: 217.2197	Val MAE: 10.1094
Epoch 79 / 100	Train MSE: 239.6224	Val MSE: 219.7691	Val MAE: 10.2872
Epoch 80 / 100	Train MSE: 238.8078	Val MSE: 218.0579	Val MAE: 10.1476
Epoch 81 / 100	Train MSE: 239.0488	Val MSE: 215.3466	Val MAE: 10.0341
Epoch 82 / 100	Train MSE: 238.3210	Val MSE: 217.7133	Val MAE: 10.1501
Epoch 83 / 100	Train MSE: 237.5547	Val MSE: 222.4161	Val MAE: 10.1801
Epoch 84 / 100	Train MSE: 239.1972	Val MSE: 217.7555	Val MAE: 10.1272
Epoch 85 / 100	Train MSE: 237.9244	Val MSE: 221.2786	Val MAE: 10.2451
Epoch 86 / 100	Train MSE: 238.5937	Val MSE: 216.2238	Val MAE: 9.9926
Epoch 87 / 100	Train MSE: 237.9108	Val MSE: 217.4094	Val MAE: 10.1039
Epoch 88 / 100	Train MSE: 238.7417	Val MSE: 216.7432	Val MAE: 10.1644
Epoch 89 / 100	Train MSE: 237.9220	Val MSE: 217.7282	Val MAE: 10.0365
Epoch 90 / 100	Train MSE: 237.5075	Val MSE: 216.8428	Val MAE: 10.1207
Epoch 91 / 100	Train MSE: 237.3194	Val MSE: 216.0090	Val MAE: 10.1310
Epoch 92 / 100	Train MSE: 237.7189	Val MSE: 218.5026	Val MAE: 10.0990
Epoch 93 / 100	Train MSE: 237.0869	Val MSE: 219.2936	Val MAE: 10.2518
Epoch 94 / 100	Train MSE: 237.1471	Val MSE: 216.6554	Val MAE: 10.0669
Epoch 95 / 100	Train MSE: 237.7732	Val MSE: 213.3517	Val MAE: 9.9985
Epoch 96 / 100	Train MSE: 237.0803	Val MSE: 216.0727	Val MAE: 10.0267
Epoch 97 / 100	Train MSE: 236.2751	Val MSE: 220.4115	Val MAE: 10.0020
Epoch 98 / 100	Train MSE: 237.4173	Val MSE: 218.1118	Val MAE: 10.0652
Epoch 99 / 100	Train MSE: 237.3875	Val MSE: 219.4919	Val MAE: 10.1987
Epoch 100 / 100	Train MSE: 236.4247	Val MSE: 216.5350	Val MAE: 10.2114

```
In [62]: # Evaluate the Model Performance on the test set
model.eval()
with torch.no_grad():
    pred_test = model(X_test_t)
    mse_test = torch.mean((pred_test - y_test_t) ** 2).item()
    mae_test = torch.mean(torch.abs(pred_test - y_test_t)).item()

    print("PyTorch Test MSE:", mse_test)
    print("PyTorch Test MAE:", mae_test)
```

```
PyTorch Test MSE: 216.53500366210938
PyTorch Test MAE: 10.211377143859863
```

```
In [63]: # Use the given features to predict the solar radio flux
x_new = np.array([[73, 2, 1, 0, 0, 0, 0, 0]])
x_new_scaled = scaler.transform(x_new)
x_new_t = torch.from_numpy(x_new_scaled).float()

# Predict using the trained model
model.eval()
with torch.no_grad():
    predicted_radio_flux = model(x_new_t).item()

    print("Predicted radio flux:", predicted_radio_flux)
```

```
Predicted radio flux: 116.77032470703125
```

```
In [64]: # Compute 95% heuristic prediction interval

margin = 1.96 * mae_test
lower_CI = predicted_radio_flux - margin
upper_CI = predicted_radio_flux + margin
print(f"95% prediction interval: [{lower_CI:.2f}, {upper_CI:.2f}]")
```

```
95% prediction interval: [96.76, 136.78]
```

Using the trained neural network model, **the predicted solar radio flux** for a day when 73 sunspots are observed on the sun and 2 solar flares with C-class energy levels and 1 solar flare with an M-class energy level occur is **approximately 116.77**. The neural network achieves **a MAE of approximately 10.21 on the test set**, which provides an empirical estimate of the prediction error on unseen data. Because neural networks do not provide statistical standard errors, a statistical prediction interval cannot be constructed as in linear regression model. Instead, the test-set MAE was used to construct **a heuristic prediction interval**, which is **roughly 96.76 to 136.78**. I am **moderately confident** in the prediction.